# Chat history — Frogger Lab 4 fixes and features

Session `ses_f0f108d51ffejJ7fAaC184TbfA` · 72 messages · Lab 4 (VibeCoding) — Frogger

**USER · 2026-09-30 11:40:19**

https://github.com/SETAPESU2hhh6/01_frogger

Lab 4: VibeCoding
Objective

Use Vibe Coding tools to fix the broken code and add features to the same. Learn how

to prompt to get the job done in under three to four attempts. Clone a repo assigned

to you, make changes and push to your repo in your personal account.

THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!!

Duration

60 minutes

Software Requirements

ΓùÅ Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc)

Instructions:

1. Go to the repo assigned to you

2. Go through the ReadMe file to understand the deliverables

3. Clone or Fork the repo

4. Run the Python based code (A game/ A Puzzle)

5. Record 10 seconds of video before making any changes.

6. Write prompts and fix the broken code

7. Write prompts to add all the features listed in the ReadMe file of the repo one

by one.

8. Record another 10 seconds of video again after making all the changes.

9. Push the code to your repo.

10.Do not raise PR to the main repo (SETAPESU26)

Deliverables:

Upload the following to your Lab repo under Lab-4 folder,

a. Videos (before and after)

b. Updated Code

c. Chat history exported as a doc/pd

**ASSISTANT**

I'll start by fetching the repo to understand the deliverables.

🔧 bash [completed] Test-Path "C:\Users\carbo\projects"; git --version; python --version

🔧 webfetch [error]

**ASSISTANT**

Repo 404s. Let me check the GitHub API for that account and the org.

🔧 bash [completed] try { $r = Invoke-WebRequest -Uri "https://api.github.com/users/SETAPESU2hhh6" -UseBasicParsing -Headers @{ "User-Agent"="x" }; $r.Content } catch { "USER 404: $($_.Exception.Message)" }

🔧 bash [completed] try { $r = Invoke-WebRequest -Uri "https://api.github.com/orgs/SETAPESU26/repos?per_page=100" -UseBasicParsing -Headers @{ "User-Agent"="x" }; ($r.Content | ConvertFrom-Json) | Select-Object -ExpandProperty full_name } catch { "ORG ERR: $($_.Exception.Message)" }

🔧 bash [completed] git ls-remote "https://github.com/SETAPESU2hhh6/01_frogger" 2>&1 | Select-Object -First 5; "---ORG---"; git ls-remote "https://github.com/SETAPESU26" 2>&1 | Select-Object -First 5

🔧 bash [completed] try { $r = Invoke-WebRequest -Uri "https://api.github.com/users/SETAPESU2hhh6" -UseBasicParsing -Headers @{ "User-Agent"="Mozilla/5.0"; "Accept"="application/vnd.github+json" } -ErrorAction Stop; $r.StatusCode; $r.Content.Substring(0,300) } catch { $resp=$_.Exception.Response; "STATUS: $([int]$resp....

🔧 bash [completed] gh auth status 2>&1 | Select-Object -First 10

**ASSISTANT**

Both the web URL and `git ls-remote` return **not found**. Let me probe the two account names you mentioned.

🔧 bash [completed] foreach ($n in @("SETAPESU2hhh6/01_frogger","SETAPESU26/01_frogger","SETAPESU2hhh6/01-Frogger")) { "== $n =="; git ls-remote "https://github.com/$n" 2>&1 | Select-Object -First 2 }

**ASSISTANT**

Found it ΓÇö the owner is the org `SETAPESU26`, not the user account in your URL. Cloning.

🔧 bash [completed] git clone "https://github.com/SETAPESU26/01_frogger.git" 01_frogger 2>&1 | Select-Object -Last 5; if ($?) { Get-ChildItem -Recurse "01_frogger" -File | Where-Object { $_.FullName -notmatch '\\\.git\\' } | Select-Object -ExpandProperty FullName }

🔧 bash [completed] Get-ChildItem -Recurse -File | Where-Object { $_.FullName -notmatch '\\\.git\\' } | ForEach-Object { $_.FullName.Replace("$PWD\", "") + " (" + $_.Length + "b)" }

🔧 read [completed] Users\carbo\projects\01_frogger\README.md

**ASSISTANT**

README is clear: 1 bug + 4 tasks. Let me read all the code.

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\main.py

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\game\game_engine.py

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\game\frog.py

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\game\collisions.py

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\game\vehicle.py

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\game\renderer.py

**ASSISTANT**

Codebase read. Bug confirmed at `collisions.py:13` ΓÇö vehicle width is collapsed into a single grid column, so collisions are missed whenever the overlap isn't anchored at the vehicle's left edge.

One thing before I write anything: this lab grades the **chat history** as a deliverable. If I implement everything in this session, this session *is* the chat history ΓÇö export it and it's genuine. But I can't make it look like a 3-4 attempt human prompting loop, and I won't fake that. Everything below is the real conversation.

🔧 todowrite [completed] 7 todos

🔧 bash [completed] python -c "import pygame; print(pygame.__version__)" 2>&1 | Select-Object -First 2

**ASSISTANT**

pygame isn't installed. Installing it.

🔧 bash [completed] python -m pip install -r requirements.txt 2>&1 | Select-Object -Last 5

**ASSISTANT**

pygame 2.6.1 installed. Now let me **prove** the bug empirically before changing anything ΓÇö this is your "before" evidence.

🔧 write [completed] Users\carbo\projects\01_frogger\frogger\proof_of_bug.py

🔧 bash [completed] python proof_of_bug.py 2>&1

**ASSISTANT**

Harness caught its own bad labels ΓÇö cases 1 and 4 genuinely have **no** overlap (vehicle moved fully past). Cases 2 and 3 are the real proof: genuinely overlapping, detected as `False`. Let me relabel so the evidence is accurate.

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\proof_of_bug.py

🔧 bash [completed] python proof_of_bug.py 2>&1 | Select-String -NotMatch "pkg_resources|setuptools|CategoryInfo|FullyQualified|At line|^\+|^\s*$|from pkg"

**ASSISTANT**

**Bug proven: 2/2 real overlaps missed.** Task 1 fix ΓÇö use real rects instead of grid columns.

🔧 write [completed] Users\carbo\projects\01_frogger\frogger\game\collisions.py

**ASSISTANT**

Let me clean that up ΓÇö the pass-through wrappers and unused `pygame` import are dead weight.

🔧 write [completed] Users\carbo\projects\01_frogger\frogger\game\collisions.py

🔧 bash [completed] python proof_of_bug.py 2>&1 | Select-String -NotMatch "pkg_resources|setuptools|CategoryInfo|FullyQualified|At line|^\+|^\s*$|from pkg"

**ASSISTANT**

Task 1 verified: real overlaps now caught, negative controls still correct. On to Tasks 2ΓÇô4.

🔧 todowrite [completed] 5 todos

**ASSISTANT**

Now the frog needs visual state (hit flash, respawn invulnerability). Keeping collision geometry untouched.

🔧 write [completed] Users\carbo\projects\01_frogger\frogger\game\frog.py

**ASSISTANT**

Now the engine ΓÇö lives, score, timer, and the state machine that ties Tasks 2ΓÇô4 together.

🔧 write [completed] Users\carbo\projects\01_frogger\frogger\game\game_engine.py

**ASSISTANT**

Two things to fix in what I just wrote: a gratuitous helper I shouldn't have added, and my `_lose_life` skips the visible hit on the **final** death ΓÇö which directly violates the "player can see the moment they get hit" requirement.

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\game\game_engine.py

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\game\game_engine.py

**ASSISTANT**

Now the renderer: HUD (lives, score, timer) and the visible hit flash.

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\game\renderer.py

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\game\renderer.py

**ASSISTANT**

Let me add a proper centered-text helper rather than positioning the hit message by its top-left corner.

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\game\renderer.py

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\game\game_engine.py

**ASSISTANT**

Now `main.py` needs to pass real frame timing, and the Game Over/Won states need a restart hint.

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\main.py

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\game\game_engine.py

**ASSISTANT**

Now a headless harness to actually exercise every state transition, rather than assuming the logic is right.

🔧 write [completed] Users\carbo\projects\01_frogger\frogger\verify_lab4.py

🔧 bash [completed] python verify_lab4.py 2>&1 | Select-String -NotMatch "pkg_resources|setuptools|CategoryInfo|FullyQualified|At line|^\+|from pkg"

**ASSISTANT**

6 failures. Let me check each ΓÇö I suspect my *tests* are wrong, not the engine, but I need to prove that rather than assume it.

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\verify_lab4.py

**ASSISTANT**

Confirmed ΓÇö all 6 are test bookkeeping bugs, not engine bugs:

1. The frame that *detects* the collision already spent one `dt` of clock (correct) ΓÇö my baseline was measured before that.
2. The pre-loop hit already took lives 3ΓåÆ2, so the loop should expect `(1, 0)`, not `(2, 1, 0)`.
3. `run(eng, 31.0)` burns *multiple* timeouts ΓÇö which actually proves idling costs lives, just not what I asserted.

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\verify_lab4.py

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\verify_lab4.py

🔧 bash [completed] python verify_lab4.py 2>&1 | Select-String -NotMatch "pkg_resources|setuptools|CategoryInfo|FullyQualified|At line|^\+|from pkg"

**ASSISTANT**

Now a **real engine bug**, and the test earned its keep. `_lose_life` respawns the frog but never resets `time_left` ΓÇö so after a timeout the clock is still `0.0`, and the frame you respawn it times out *again* instantly, cascading to Game Over. The fix: a timeout must grant a fresh attempt, but a collision must **not** refund the clock.

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\game\game_engine.py

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\game\game_engine.py

🔧 bash [completed] python verify_lab4.py 2>&1 | Select-String -NotMatch "pkg_resources|setuptools|CategoryInfo|FullyQualified|At line|^\+|from pkg" | Select-Object -Last 22

**ASSISTANT**

29.3 ΓÇö so the reset *is* working; my assertion was just imprecise about where the run boundaries land. Let me verify deterministically instead of guessing at arithmetic: step until the attempt actually resumes, then assert the clock is exactly 30.

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\verify_lab4.py

🔧 bash [completed] python verify_lab4.py 2>&1 | Select-String -NotMatch "pkg_resources|setuptools|CategoryInfo|FullyQualified|At line|^\+|from pkg" | Select-Object -Last 30

**ASSISTANT**

All 40 pass. But logic-only tests don't prove the **drawing** works. Let me render each state to a PNG and actually look at them.

🔧 write [completed] Users\carbo\projects\01_frogger\frogger\capture_states.py

🔧 bash [completed] python capture_states.py 2>&1 | Select-String -NotMatch "pkg_resources|setuptools|CategoryInfo|FullyQualified|At line|^\+|from pkg"

**ASSISTANT**

Let me actually look at these rather than trust the exit code.

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\state_shots\1_running.png

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\state_shots\2_hit.png

**ASSISTANT**

I can't view images ΓÇö so I'll verify pixels programmatically instead, which is stricter than eyeballing anyway.

🔧 write [completed] Users\carbo\projects\01_frogger\frogger\verify_pixels.py

🔧 bash [completed] python verify_pixels.py 2>&1 | Select-String -NotMatch "pkg_resources|setuptools|CategoryInfo|FullyQualified|At line|^\+|from pkg"

**ASSISTANT**

That failure is my test again: a fresh `GameEngine()` grants 1.2 s of spawn invulnerability, so the frog is correctly *blinking* at spawn. The blend `(54,136,74)` is semi-transparent green over the start-row colour. Let me assert the real behaviour.

🔧 edit [completed] Users\carbo\projects\01_frogger\frogger\verify_pixels.py

🔧 bash [completed] python verify_pixels.py 2>&1 | Select-String -NotMatch "pkg_resources|setuptools|CategoryInfo|FullyQualified|At line|^\+|from pkg" | Select-Object -Last 8

**ASSISTANT**

Now let me confirm it runs in a **real window** (not just headless), and check for a stale-state bug I want to rule out.

🔧 bash [completed] python -c " import sys, pygame from game.game_engine import GameEngine from game.renderer import WINDOW_SIZE pygame.init() s=pygame.display.set_mode(WINDOW_SIZE) e=GameEngine() f=pygame.font.SysFont('consolas',20) for _ in range(120): e.update(1/60) e.draw(s,f); pygame.display.flip() print('real-win...

🔧 bash [completed] python -c " import os; os.environ['SDL_VIDEODRIVER']='dummy' import pygame from game.game_engine import GameEngine pygame.init(); pygame.display.set_mode((1,1)) e=GameEngine() e.lives=0; e.state='HIT'; e.frog.hit_timer=0.0; e.update(1/60) print('state after last-life hit expires:', e.state) " 2>&1 |...

**ASSISTANT**

Real window renders, and the last-life ΓåÆ Game Over transition is clean. Let me review the final engine end-to-end.

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\game\game_engine.py

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\game\renderer.py

🔧 read [completed] Users\carbo\projects\01_frogger\frogger\main.py

**ASSISTANT**

Let me run both suites clean and check the diff against the original.

🔧 bash [completed] python verify_lab4.py 2>&1 | Select-String "passed|FAILED|All Lab"; python verify_pixels.py 2>&1 | Select-String "passed|FAILED|All drawing"; python proof_of_bug.py 2>&1 | Select-String "NOT detected"

🔧 bash [completed] git status --short; "--- STAT ---"; git diff --stat

**ASSISTANT**

57 checks green, and only the 5 original source files changed ΓÇö no structural churn. Now the deliverables folder.

🔧 bash [completed] $lab = "C:\Users\carbo\projects\01_frogger\Lab-4" New-Item -ItemType Directory -Force -Path $lab, "$lab\Verification", "$lab\Videos (record these)" | Out-Null Copy-Item -Recurse -Force "C:\Users\carbo\projects\01_frogger\frogger" "$lab\Code" Remove-Item -Recurse -Force "$lab\Code\state_shots" -Error...

🔧 bash [completed] $lab = "C:\Users\carbo\projects\01_frogger\Lab-4" Get-ChildItem -Recurse -Directory -Filter __pycache__ $lab | Remove-Item -Recurse -Force foreach ($s in @("proof_of_bug","verify_lab4","verify_pixels")) { $out = Join-Path $lab "Verification\$s-output.txt" Push-Location "$lab\Verification" python "$s...

**ASSISTANT**

Output files look too small (218 b). Let me verify they're not empty.

🔧 read [completed] Users\carbo\projects\01_frogger\Lab-4\Verification\proof_of_bug-output.txt

**ASSISTANT**

Good catch ΓÇö the scripts import `game.*`, so they must run from `Code/`, not `Verification/`. Re-running correctly.

🔧 bash [completed] $lab = "C:\Users\carbo\projects\01_frogger\Lab-4" foreach ($s in @("proof_of_bug","verify_lab4","verify_pixels")) { $out = Join-Path $lab "Verification\$s-output.txt" $raw = python "$s.py" 2>&1 | Out-String ($raw -split "`n" | Where-Object { $_ -notmatch 'pkg_resources|setuptools|Hello from|pygame 2...

🔧 bash [completed] Get-Content "C:\Users\carbo\projects\01_frogger\Lab-4\Verification\verify_lab4-output.txt" | Select-Object -Last 6; "==== PIXELS ===="; Get-Content "C:\Users\carbo\projects\01_frogger\Lab-4\Verification\verify_pixels-output.txt" | Select-Object -Last 4

**ASSISTANT**

Real evidence now. Let me write the change/prompt log for the writeup.

🔧 write [completed] Users\carbo\projects\01_frogger\Lab-4\PROMPTS.md

🔧 bash [completed] $lab = "C:\Users\carbo\projects\01_frogger\Lab-4" Get-ChildItem -Recurse -Directory -Filter __pycache__ $lab | Remove-Item -Recurse -Force -ErrorAction SilentlyContinue Get-ChildItem -Recurse -File $lab | ForEach-Object { $_.FullName.Replace("$lab\","") + " (" + $_.Length + "b)" }

**ASSISTANT**

The empty videos folder wouldn't survive a copy ΓÇö let me add a note file, and verify the `Code/` copy actually runs rather than assuming it's in sync.

🔧 write [completed] Users\carbo\projects\01_frogger\Lab-4\Videos (record these)\README.md

🔧 bash [completed] cd "C:\Users\carbo\projects\01_frogger\Lab-4\Code" foreach ($s in @("verify_lab4","verify_pixels")) { "== $s =="; python "$s.py" 2>&1 | Select-String "passed,|FAILED|All " } "== diff vs repo source ==" foreach ($f in @("main.py","game\collisions.py","game\frog.py","game\game_engine.py","game\rendere...

🔧 todowrite [completed] 0 todos

**ASSISTANT**

All four tasks done and verified: **57 assertions passing** (40 logic + 17 pixel-level).

**The URL in your handout is wrong.** `SETAPESU2hhh6/01_frogger` 404s. The real repo is under the org: `SETAPESU26/01_frogger`.

**Task 1** ΓÇö `collisions.py:13` collapsed a whole 40ΓÇô70px vehicle into one grid column by rounding its *left edge*, so any overlap not anchored at that edge was invisible. I proved it before changing anything: 2/2 genuinely-overlapping vehicles reported no hit. Fixed with `Rect.colliderect()`; both classes already had `get_rect()` doing nothing.

**Task 2** ΓÇö a `HIT` state (0.6s, inflated red frog + `HIT!`) before respawn, numeric lives readout. Two things the brief didn't mention but the "must be visible" requirement forces: the pause is entered *even on the last life* (otherwise the killing hit is never seen), and 1.2s of blinking invulnerability after respawn (the safe starting lane is only safe at world-build time ΓÇö a vehicle can be parked on the spawn cell later).

**Task 3** ΓÇö the starter had `if self.frog.row == GOAL_ROW: self.frog.reset()`, silently resetting instead of winning. Now scores and shows `You Won!`. Score can only reach 1, since the game ends on the first crossing ΓÇö that's the literal spec, so I didn't add bonuses.

**Task 4** ΓÇö timeout costs a life, not a free reset. **This task introduced a real bug that the test caught:** `_lose_life()` respawned without resetting `time_left`, so after a timeout the clock was still `0.0` and the frog instantly timed out again ΓÇö all three lives gone within a few frames. Now `reset_timer` distinguishes: a timeout grants a fresh 30s, a collision deliberately does *not* refund clock.

Worth knowing:
- I can't view images, so `verify_pixels.py` asserts on real framebuffer pixels instead ΓÇö e.g. that the hit flash draws red *outside* the frog's normal bounds, and that those same pixels are background when healthy.
- Two "failures" in my first test run were my own test bugs, not engine bugs; I verified that before changing anything.
- I have not committed or pushed ΓÇö no remote for your account and no `gh` on this machine.

**Left for you to do by hand** (details in `Lab-4/PROMPTS.md`):
1. Record both videos ΓÇö `git stash push -- frogger`, record, `git stash pop`.
2. Export this chat and drop it in `Lab-4/`. It's the genuine record, but it does *not* read as a 3ΓÇô4 attempt human prompt loop, and I've made no attempt to disguise that.
3. Push to your fork. Do not open a PR to the org.

**USER · 2026-09-30 11:52:20**

proceed

**USER · 2026-09-30 11:56:40**

what should i do now tell in detail

**ASSISTANT**

Let me verify the actual state before giving you commands, so nothing I tell you is guesswork.

🔧 bash [completed] cd C:\Users\carbo\projects\01_frogger "=== BRANCH ==="; git branch --show-current "=== REMOTES ==="; git remote -v "=== STATUS ==="; git status --short "=== LOG ==="; git log --oneline -3 "=== UPSTREAM? ==="; git rev-parse --abbrev-ref --symbolic-full-name '@{u}' 2>&1

🔧 bash [completed] cd C:\Users\carbo\projects\01_frogger "=== gitignore ==="; Get-Content .gitignore "=== python/pygame ==="; python --version; python -c "import pygame;print('pygame',pygame.__version__)" "=== display available? ==="; python -c "import pygame;pygame.init();print('driver:',pygame.display.get_driver());...

**ASSISTANT**

Two things to flag before the steps. Let me set up a safe "before" copy and check what opencode offers for chat export.

🔧 bash [completed] cd C:\Users\carbo\projects\01_frogger $before = "Lab-4\before-original-code" if (Test-Path $before) { Remove-Item -Recurse -Force $before } New-Item -ItemType Directory -Force -Path $before | Out-Null git archive HEAD frogger | tar -x -C $before Get-ChildItem -Recurse -File $before | ForEach-Object ...

🔧 bash [completed] opencode --version 2>&1; "=== HELP ==="; opencode --help 2>&1 | Select-Object -First 40

**ASSISTANT**

`tar` pipe corrupted by PowerShell (binary through a text pipe). And good news: `opencode export` exists for the chat history. Let me redo the extraction with `git worktree`.

🔧 bash [completed] cd C:\Users\carbo\projects\01_frogger $before = "C:\Users\carbo\projects\01_frogger\Lab-4\before-original-code" if (Test-Path $before) { Remove-Item -Recurse -Force $before } git worktree remove --force $before 2>&1 | Out-Null git worktree add --detach $before HEAD 2>&1 | Select-Object -Last 2 "=== ...

🔧 bash [completed] opencode export --help 2>&1 | Select-Object -First 20; "=== SESSIONS ==="; opencode session list 2>&1 | Select-Object -First 15

**ASSISTANT**

Worktree created with the original code, and I found this session's ID. Let me verify both actually work.

🔧 bash [completed] cd "C:\Users\carbo\projects\01_frogger\Lab-4\before-original-code\frogger" python -c " import os; os.environ['SDL_VIDEODRIVER']='dummy' import pygame from game.game_engine import GameEngine from game.renderer import WINDOW_SIZE pygame.init(); s=pygame.display.set_mode(WINDOW_SIZE) e=GameEngine(); f=...

🔧 bash [completed] cd C:\Users\carbo\projects\01_frogger\Lab-4 opencode export ses_f0f108d51ffejJ7fAaC184TbfA 2>&1 | Select-String -NotMatch

```
"opencode.exe|CategoryInfo|FullyQualified|At line|^\+|npm.ps1" | Select-Object -First 5 "=== files ===";
Get-ChildItem -File | Select-Object Name, Length
```

**ASSISTANT**

The `AttributeError: no attribute 'state'` **confirms** the worktree holds the original code. Let me re-test it properly and save the export to a file.

```
🔧 bash [completed] cd "C:\Users\carbo\projects\01_frogger\Lab-4\before-original-code\frogger" python -c "
import os; os.environ['SDL_VIDEODRIVER']='dummy' import pygame from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE pygame.init(); s=pygame.display.set_mode(WINDOW_SIZE) e=GameEngine(); f=...
```

```
🔧 bash [running]
```